Sesión 5: Evaluación de Modelos

¿Qué tan bueno es mi modelo?

Agustín Gurvich

Introducción a Machine Learning

Lo que ya vimos

- Cuatro clasificadores: logística, árboles, kNN y SVM
- Para saber si andaban hicimos siempre lo mismo: `modelo.score(X_test, y_test)` sobre **un** test set

Lo que ya vimos

- Cuatro clasificadores: logística, árboles, kNN y SVM
- Para saber si andaban hicimos siempre lo mismo: `modelo.score(X_test, y_test)` sobre **un** test set

Hoy vamos a ver:

1. ¿El accuracy alcanza para saber si un modelo es bueno?
2. ¿Podemos confiar en el resultado de un único split?
3. ¿Cómo elegimos hiperparámetros?

Parte I

Métricas

El detector de fraudes

Un banco tiene 10.000 transacciones, de las cuales **200 son fraudes** (el 2 %). Armamos un detector de fraude.

```
def detector(transaccion):
    return "no es fraude"
```

El detector de fraudes

Un banco tiene 10.000 transacciones, de las cuales **200 son fraudes** (el 2 %). Armamos un detector de fraude.

```
def detector(transaccion):
    return "no es fraude"
```

Accuracy: $\frac{9800}{10000} = 98\,\%$

Importante

98 % de accuracy y no detecta ni un solo fraude!!! Un número alto de accuracy no garantiza que el modelo sirva para algo.

Clases desbalanceadas

Definición

Un problema tiene clases desbalanceadas cuando una de las clases es mucho más frecuente que la otra.

Y suelen ser los problemas más interesantes:

- Fraudes con tarjeta de crédito
- Enfermedades poco frecuentes
- Fallas en una máquina de una fábrica
- Clientes que se van a dar de baja de un servicio

Clases desbalanceadas

Definición

Un problema tiene clases desbalanceadas cuando una de las clases es mucho más frecuente que la otra.

Y suelen ser los problemas más interesantes:

- Fraudes con tarjeta de crédito
- Enfermedades poco frecuentes
- Fallas en una máquina de una fábrica
- Clientes que se van a dar de baja de un servicio

Ejemplo

Antes de decir que mi accuracy es buena, hay que comparalo contra el modelo más dummy posible: siempre predecir la clase más frecuente. Si no le gana por bastante, no hay aprendizaje del modelo.

Positivos y negativos

- En clasificación binaria, a una clase la llamamos positiva (la que estamos buscando): fraude, tumor maligno, spam
- "Positivo" no es "bueno", es "lo que queremos detectar".

Positivos y negativos

- En clasificación binaria, a una clase la llamamos positiva (la que estamos buscando): fraude, tumor maligno, spam
- "Positivo" no es "bueno", es "lo que queremos detectar".

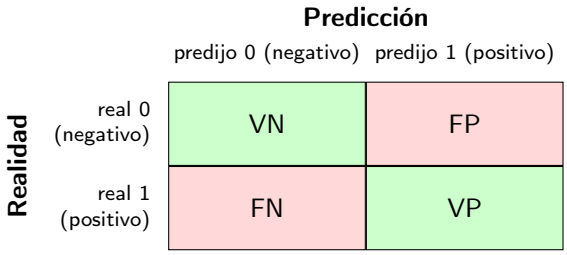
Cada predicción cae en uno de cuatro casos:

	¿Qué pasó?	Ejemplo
VP (verdadero positivo)	dijo positivo y era positivo	detectó un fraude
VN (verdadero negativo)	dijo negativo y era negativo	dejó pasar una compra normal
FP (falso positivo)	dijo positivo y era negativo	bloqueó una compra normal
FN (falso negativo)	dijo negativo y era positivo	se le escapó un fraude

La matriz de confusión

Definición

La matriz de confusión cuenta cuántas predicciones cayeron en cada uno de los cuatro casos.



- La diagonal son los aciertos. Fuera de la diagonal están los errores
- Es el mismo orden que usa scikit-learn: filas = realidad, columnas = predicción

Los dos errores no cuestan lo mismo

Falso positivo: una falsa alarma

- Un mail importante termina en spam
- Le bloquean la tarjeta a alguien que estaba comprando
- A un paciente sano le piden más estudios

Falso negativo: algo que se escapó

- Un spam llega a la bandeja de entrada
- Pasa una compra con una tarjeta robada
- No se detecta un tumor maligno

Los dos errores no cuestan lo mismo

Falso positivo: una falsa alarma

- Un mail importante termina en spam
- Le bloquean la tarjeta a alguien que estaba comprando
- A un paciente sano le piden más estudios

Falso negativo: algo que se escapó

- Un spam llega a la bandeja de entrada
- Pasa una compra con una tarjeta robada
- No se detecta un tumor maligno

Importante

El accuracy cuenta todos los errores igual, pero en casi todos los problemas reales uno de los dos errores es mucho más grave que el otro. Cuál es más grave depende del problema, no del modelo.

Un ejemplo con números

1000 transacciones, 50 fraudes. Comparamos dos detectores:

Modelo A

	pred. 0	pred. 1
real 0	890	60
real 1	10	40

Accuracy: $\frac{890+40}{1000} = 93\,\%$

Dummy (siempre "no")

	pred. 0	pred. 1
real 0	950	0
real 1	50	0

Accuracy: $\frac{950}{1000} = 95\,\%$

Un ejemplo con números

1000 transacciones, 50 fraudes. Comparamos dos detectores:

Modelo A

	pred. 0	pred. 1
real 0	890	60
real 1	10	40

Accuracy: $\frac{890+40}{1000} = 93\,\%$

Dummy (siempre "no")

	pred. 0	pred. 1
real 0	950	0
real 1	50	0

Accuracy: $\frac{950}{1000} = 95\,\%$

El dummy tiene más accuracy, pero el modelo A encuentra 40 de los 50 fraudes. Necesitamos métricas que miren a la clase positiva.

Precision

Definición

$$\text{Precision} = \frac{VP}{VP + FP}$$

De todos los casos que el modelo marcó como positivos, ¿cuántos lo eran de verdad?

- Modelo A: marcó $40 + 60 = 100$ transacciones como fraude, y 40 lo eran $\Rightarrow$ precision $= 0{,}40$
- Precision baja = muchas falsas alarmas. De cada 10 tarjetas que bloquea, 6 eran de clientes que no hicieron nada malo

Recall

Definición

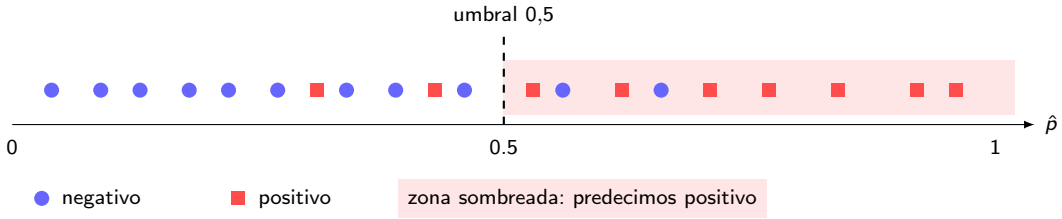
$$\text{Recall} = \frac{VP}{VP + FN}$$

De todos los positivos que había, ¿cuántos encontró el modelo?

- Modelo A: había 50 fraudes y encontró 40 $\Rightarrow$ recall $= 0{,}80$
- dummy: había 50 y encontró 0 $\Rightarrow$ recall $= 0$
- Recall bajo = se escapan muchos positivos

El umbral otra vez

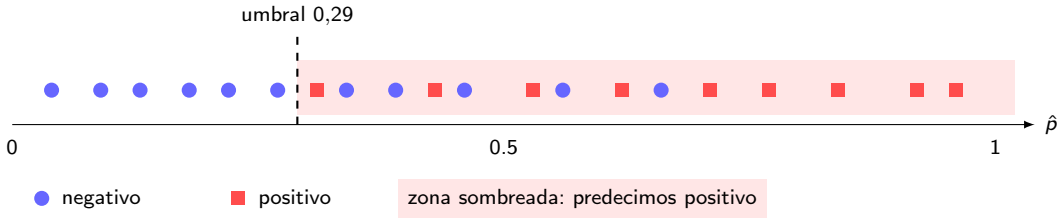
La regresión logística (y casi todos los clasificadores) dan una probabilidad. El umbral decide a partir de cuánto decimos "positivo".



Umbral 0,5: VP $= 7$, FP $= 2$, FN $= 2$ $\Rightarrow$ precision $= \frac{7}{9} \approx 0{,}78$, recall $= \frac{7}{9} \approx 0{,}78$

El umbral otra vez

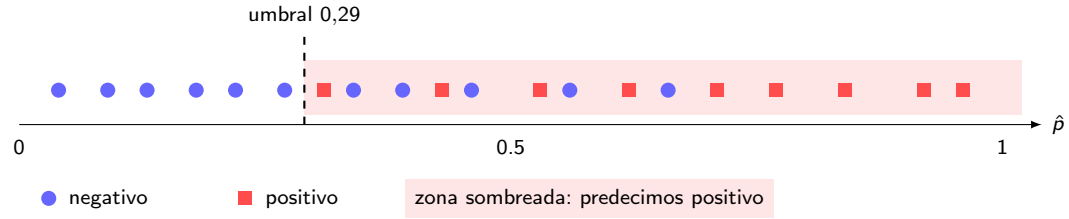
La regresión logística (y casi todos los clasificadores) dan una probabilidad. El umbral decide a partir de cuánto decimos "positivo".



Umbral 0,29: VP $= 9$, FP $= 5$, FN $= 0$ $\Rightarrow$ precision $= \frac{9}{14} \approx 0{,}64$, recall $= \frac{9}{9} = 1$

El umbral otra vez

La regresión logística (y casi todos los clasificadores) dan una probabilidad. El umbral decide a partir de cuánto decimos "positivo".



Umbral 0,29: VP $= 9$, FP $= 5$, FN $= 0$ $\Rightarrow$ precision $= \frac{9}{14} \approx 0{,}64$, recall $= \frac{9}{9} = 1$

Importante

Bajar el umbral sube el recall y baja la precision. Subirlo hace lo contrario. Elegir el umbral es elegir qué error preferimos cometer.

¿Precision o recall?

Problema	Lo más grave	Priorizar
Detectar un tumor maligno	que se escape un enfermo (FN)	recall
Filtro de spam	que un mail importante vaya a spam (FP)	precision
Detectar fraudes	que pase un fraude (FN)	recall
Recomendar videos	recomendar algo que no te interesa (FP)	precision

¿Precision o recall?

Problema	Lo más grave	Priorizar
Detectar un tumor maligno	que se escape un enfermo (FN)	recall
Filtro de spam	que un mail importante vaya a spam (FP)	precision
Detectar fraudes	que pase un fraude (FN)	recall
Recomendar videos	recomendar algo que no te interesa (FP)	precision

Importante

Un modelo que dice "positivo" para todo tiene recall $= 1$. Uno que dice "positivo" una sola vez y acierta tiene precision $= 1$. Por eso nunca hay que ver una sola de las dos.

F1: las dos en un solo número

Definición

El **F1** es la media armónica entre precision y recall:

$$F_1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

¿Por qué no el promedio común? Un modelo con precision $= 1$ y recall $= 0{,}02$ (encontró 1 fraude de 50):

- Promedio común: $\frac{1+0{,}02}{2} = 0{,}51$ (parece mediocre)

F1: las dos en un solo número

Definición

El **F1** es la media armónica entre precision y recall:

$$F_1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

¿Por qué no el promedio común? Un modelo con precision $= 1$ y recall $= 0{,}02$ (encontró 1 fraude de 50):

- Promedio común: $\frac{1+0{,}02}{2} = 0{,}51$ (parece mediocre)
- F1: $2 \cdot \frac{1 \cdot 0{,}02}{1+0{,}02} \approx 0{,}04$ (es malísimo, y el F1 lo dice)

F1: las dos en un solo número

Definición

El **F1** es la media armónica entre precision y recall:

$$F_1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

¿Por qué no el promedio común? Un modelo con precision $= 1$ y recall $= 0{,}02$ (encontró 1 fraude de 50):

- Promedio común: $\frac{1+0{,}02}{2} = 0{,}51$ (parece mediocre)
- F1: $2 \cdot \frac{1 \cdot 0{,}02}{1+0{,}02} \approx 0{,}04$ (es malísimo, y el F1 lo dice)

El F1 solo es alto si las dos son altas. Modelo A: $F_1 = 2 \cdot \frac{0{,}4 \cdot 0{,}8}{0{,}4+0{,}8} \approx 0{,}53$

Resumen de las métricas

Métrica	Fórmula	¿Qué responde?	Modelo A
Accuracy	$\frac{VP + VN}{VP + VN + FP + FN}$	¿Qué proporción de todo acertó?	0,93
Precision	$\frac{VP}{VP + FP}$	De lo que marcó positivo, ¿cuánto lo era?	0,40
Recall	$\frac{VP}{VP + FN}$	De los positivos, ¿cuántos encontró?	0,80
F1	$2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$	¿Están las dos equilibradas y altas?	0,53

Modelo A: VP $= 40$, FP $= 60$, FN $= 10$, VN $= 890$.

Más de dos clases

Con n clases la matriz de confusión es de $n \times n$. Por ejemplo, con iris:

	pred. setosa	pred. versicolor	pred. virginica
real setosa	**15**	0	0
real versicolor	0	**13**	2
real virginica	0	1	**14**

- Muestra **entre qué clases** se confunde el modelo: setosa nunca, versicolor y virginica a veces
- Precision y recall se calculan para cada clase ("esta clase" vs. "todas las demás") y después se promedian (macro average).

Métricas con scikit-learn

```
from sklearn.metrics import (accuracy_score, precision_score,
    recall_score, f1_score, confusion_matrix,
    ConfusionMatrixDisplay, classification_report)

y_pred = modelo.predict(X_test)

confusion_matrix(y_test, y_pred)
precision_score(y_test, y_pred)      # de la clase 1
recall_score(y_test, y_pred)
f1_score(y_test, y_pred)

print(classification_report(y_test, y_pred))    # todo junto
ConfusionMatrixDisplay.from_predictions(y_test, y_pred)
```

scikit-learn considera positiva a la clase 1. Hay que fijarse que la clase que buscamos sea la 1!

¿Y en regresión?

Cuando predecimos un número no hay aciertos y errores, hay errores más grandes o más chicos. Si y_i es el valor real y $\hat{y}_i$ la predicción:

- **MAE** (error absoluto medio): $\frac{1}{n}\sum |y_i - \hat{y}_i|$. "En promedio le erramos por tanto"
- **RMSE** (raíz del error cuadrático medio): $\sqrt{\frac{1}{n}\sum (y_i - \hat{y}_i)^2}$. Castiga más los errores grandes
- R^2: el que vimos en la sesión 3. Qué proporción de la variación de y explica el modelo

¿Y en regresión?

Cuando predecimos un número no hay aciertos y errores, hay errores más grandes o más chicos. Si y_i es el valor real y $\hat{y}_i$ la predicción:

- **MAE** (error absoluto medio): $\frac{1}{n}\sum |y_i - \hat{y}_i|$. "En promedio le erramos por tanto"
- **RMSE** (raíz del error cuadrático medio): $\sqrt{\frac{1}{n}\sum (y_i - \hat{y}_i)^2}$. Castiga más los errores grandes
- R^2: el que vimos en la sesión 3. Qué proporción de la variación de y explica el modelo

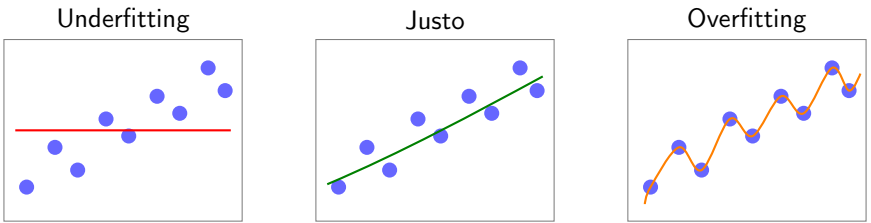
Ejemplo

Prediciendo precios de departamentos, un MAE de 5.000 dólares se entiende enseguida. Un $R^2 = 0{,}83$ es más difícil de explicar.

Parte II

Overfitting y validación

Lo que ya vimos

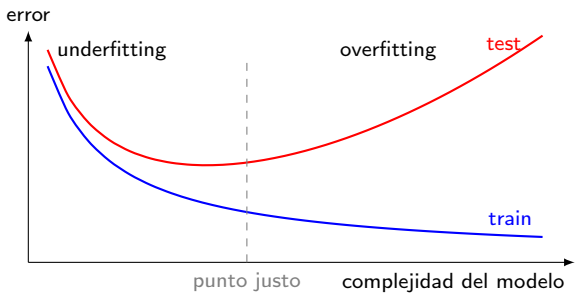


- **Underfitting**: el modelo es demasiado simple, le va mal hasta en train
- **Overfitting**: el modelo memoriza los datos de train, ruido incluido, y le va mal con datos nuevos
- Lo controlamos con hiperparámetros: `max_depth` en árboles, k en kNN, C en SVM

Generalizar

Definición

Un modelo **generaliza** bien cuando le va bien con datos que nunca vio. Es lo más importante: el modelo se va a usar con datos nuevos.



Conclusión

Train	Test	Conclusión
malo	malo	underfitting
muy bueno	bastante peor	overfitting
bueno	parecido	¡bien!

Conclusión

Train	Test	Conclusión
malo	malo	underfitting
muy bueno	bastante peor	overfitting
bueno	parecido	¡bien!

Si hay underfitting

- Modelo más flexible
- Más o mejores features

Si hay overfitting

- Modelo más simple
- Conseguir más datos
- Sacar features que no aportan

La trampa que hicimos la clase pasada

En la sesión 4 elegimos `max_depth` probando muchos valores y quedándonos con el que mejor andaba en test.

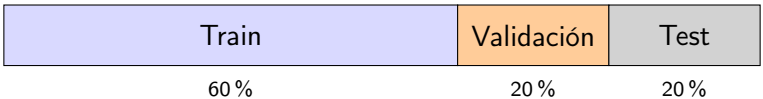
La trampa que hicimos la clase pasada

En la sesión 4 elegimos `max_depth` probando muchos valores y quedándonos con el que mejor andaba en test.

Importante

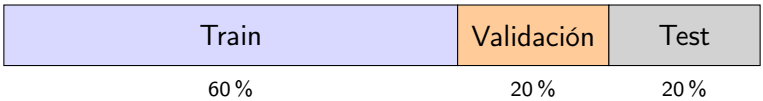
Si usamos el test set para tomar decisiones, deja de ser un test: el resultado que da es más optimista que la realidad. El test set se mira una sola vez, al final.

Tres conjuntos



- Train: para entrenar
- Validación: para comparar modelos y elegir hiperparámetros
- Test: para medir, una sola vez, qué tan bueno es el modelo que elegimos

Tres conjuntos



- Train: para entrenar
- Validación: para comparar modelos y elegir hiperparámetros
- Test: para medir, una sola vez, qué tan bueno es el modelo que elegimos

Dos problemas:

- Nos quedan menos datos para entrenar
- Con datasets chicos, el resultado depende mucho de la suerte del reparto

Un solo split es una lotería

Mismo árbol (`max_depth=4`), mismo dataset (cáncer de mama), solo cambiamos el `random_state` del `train_test_split`:

`random_state`	0	1	2	3	...
accuracy en test	0.92	0.92	0.95	0.91	

Un solo split es una lotería

Mismo árbol (`max_depth=4`), mismo dataset (cáncer de mama), solo cambiamos el `random_state` del `train_test_split`:

`random_state`	0	1	2	3	...
accuracy en test	0.92	0.92	0.95	0.91	

Probando 30 splits distintos, el accuracy va de **0.88** a **0.96**.

Un solo split es una lotería

Mismo árbol (`max_depth=4`), mismo dataset (cáncer de mama), solo cambiamos el `random_state` del `train_test_split`:

`random_state`	0	1	2	3	...
accuracy en test	0.92	0.92	0.95	0.91	

Probando 30 splits distintos, el accuracy va de **0.88** a **0.96**.

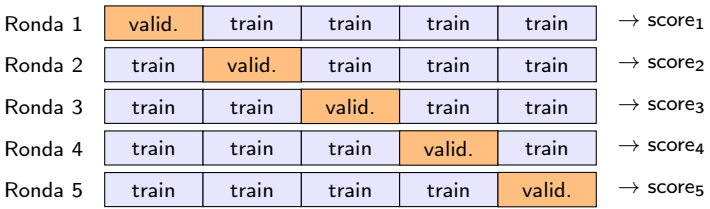
Importante

Si comparamos dos modelos con un solo split, la diferencia entre ellos puede ser pura suerte. Necesitamos una medida más estable.

Validación cruzada (k-fold)

Definición

Partimos los datos en k partes iguales (**folds**). Entrenamos k veces: cada vez usamos una parte distinta para validar y las demás para entrenar. El resultado es el promedio de las k evaluaciones.



Resultado final: promedio (y desvío) de los 5 scores.

¿Por qué funciona mejor?

- Todos los datos se usan para validar exactamente una vez, y para entrenar $k - 1$ veces
- El promedio de k resultados es mucho más estable que un solo resultado
- El desvío nos dice cuánto podemos confiar: $0{,}92 \pm 0{,}01$ no es lo mismo que $0{,}92 \pm 0{,}08$

¿Por qué funciona mejor?

- Todos los datos se usan para validar exactamente una vez, y para entrenar $k - 1$ veces
- El promedio de k resultados es mucho más estable que un solo resultado
- El desvío nos dice cuánto podemos confiar: $0{,}92 \pm 0{,}01$ no es lo mismo que $0{,}92 \pm 0{,}08$
- Valores típicos: $k = 5$ o $k = 10$
- El costo: entrenar el modelo k veces
- En clasificación, scikit-learn hace los folds estratificados, es deicr, cada fold tiene la misma proporción de clases que el dataset completo

Validación cruzada con scikit-learn

```
from sklearn.model_selection import cross_val_score

modelo = DecisionTreeClassifier(max_depth=4)
scores = cross_val_score(modelo, X_train, y_train, cv=5)

scores            # array con 5 accuracies
scores.mean()     # el promedio
scores.std()      # el desvio

# con otra metrica
cross_val_score(modelo, X_train, y_train, cv=5, scoring="recall")
```

cross_val_score hace todo: parte, entrena k veces y evalúa.

Eligiendo hiperparámetros: GridSearchCV

Probar todas las combinaciones de hiperparámetros, cada una con validación cruzada, y quedarse con la mejor:

```
from sklearn.model_selection import GridSearchCV

grilla = {"max_depth": [2, 3, 4, 5, 8],
          "min_samples_leaf": [1, 5, 10]}

busqueda = GridSearchCV(DecisionTreeClassifier(), grilla, cv=5)
busqueda.fit(X_train, y_train)          # 5 x 3 x 5 = 75
    entrenamientos

busqueda.best_params_                   # la mejor combinacion
busqueda.score(X_test, y_test)          # recien ahora, el test
```

El flujo completo

1. Separar el test set y no tocarlo

El flujo completo

1. Separar el test set y no tocarlo
2. Con el train, usar validación cruzada para comparar modelos y elegir hiperparámetros

El flujo completo

1. Separar el test set y no tocarlo
2. Con el train, usar validación cruzada para comparar modelos y elegir hiperparámetros
3. Entrenar el modelo elegido con todo el train (`GridSearchCV` lo hace solo)

El flujo completo

1. Separar el test set y no tocarlo
2. Con el train, usar validación cruzada para comparar modelos y elegir hiperparámetros
3. Entrenar el modelo elegido con todo el train (`GridSearchCV` lo hace solo)
4. Evaluar una vez en test. Ese es el número que reportamos

El flujo completo

1. Separar el test set y no tocarlo
2. Con el train, usar validación cruzada para comparar modelos y elegir hiperparámetros
3. Entrenar el modelo elegido con todo el train (`GridSearchCV` lo hace solo)
4. Evaluar una vez en test. Ese es el número que reportamos

Importante

Si después de mirar el test volvemos al paso 2 a cambiar cosas, el test deja de ser confiable.

Filtrar información del futuro

Definición

Hay **data leakage** (fuga de datos) cuando información de los datos de validación o test se mete, sin querer, en el entrenamiento. El resultado parece excelente y en la vida real el modelo no funciona.

Ejemplos típicos:

- Ajustar el `StandardScaler` con todos los datos antes de separar (ya lo vimos en la sesión 3)
- Elegir las "mejores features" mirando todo el dataset, y recién después hacer validación cruzada
- Usar una feature que en la vida real no vamos a tener al momento de predecir. Por ejemplo, para predecir si un paciente tiene una enfermedad, usar "recibió tratamiento para esa enfermedad"

La solución: pipelines

Todo lo que aprende de los datos (escalar, elegir features, etc.) tiene que ir adentro de la validación cruzada. Un `pipeline` lo hace solo:

```
from sklearn.pipeline import make_pipeline

modelo = make_pipeline(StandardScaler(), KNeighborsClassifier())
cross_val_score(modelo, X_train, y_train, cv=5)
```

En cada iteración, el scaler se ajusta solo con los folds de train de esa ronda.